

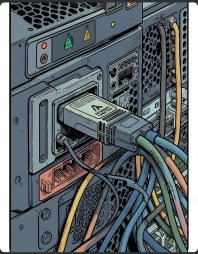
Conexión a Base de Datos desde C#/.NET

SqlConnection, SqlCommand y SqlDataReader — fundamentos para acceder a SQL Server desde aplicaciones .NET.

¿Qué es ADO.NET y por qué importa?

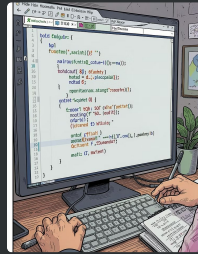
ADO.NET es el conjunto de clases de .NET para conectarse, ejecutar consultas y manipular datos en SGBD (ej. SQL Server). Proporciona una forma estandarizada, segura y eficiente de interactuar con bases de datos, y es la base de tecnologías superiores (Entity Framework, Dapper).

Componentes clave



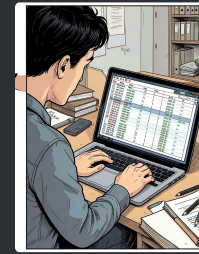
SqlConnection

Administra la conexión con SQL Server (cadena de conexión, recursos). Usar using para liberar recursos automáticamente.



SqlCommand

Representa la instrucción SQL; ejecuta SELECT/INSERT/UPDATE/DELETE o procedimientos almacenados mediante ExecuteReader, ExecuteNonQuery o ExecuteScalar.



SqlDataReader

Lectura secuencial, forward-only y read-only. Lee fila a fila reduciendo uso de memoria; depende de la conexión abierta.



Métodos de ejecución de SqlCommand

1

ExecuteReader()

Devuelve un SqlDataReader para consultas que retornan filas (SELECT).

2

ExecuteNonQuery()

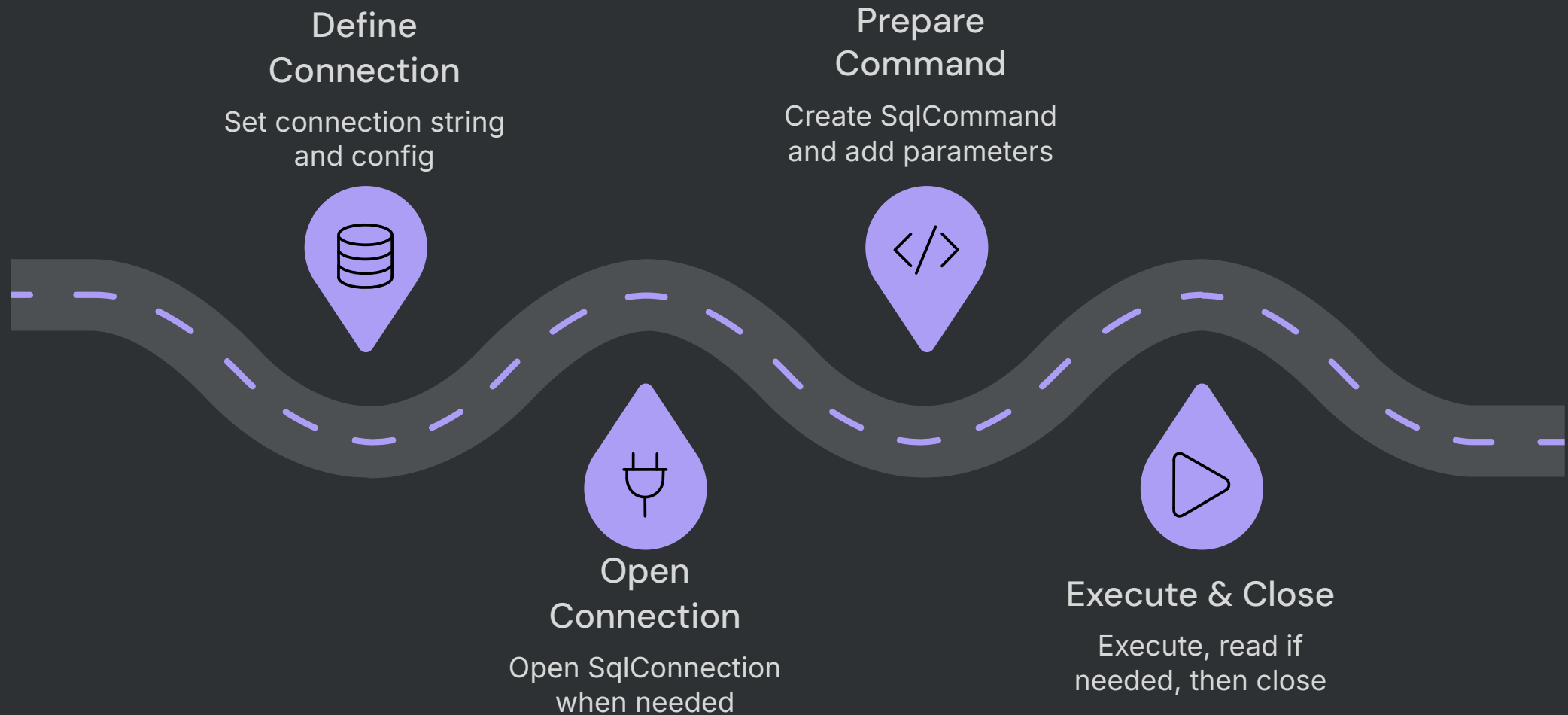
Para operaciones que modifican datos (INSERT/UPDATE/DELETE). Retorna número de filas afectadas.

3

ExecuteScalar()

Cuando se espera un único valor (ej. COUNT, MAX). Retorna un object.

Flujo de trabajo — pasos típicos



Resumen: abrir conexión justo antes, ejecutar con parámetros y cerrar inmediatamente para aprovechar connection pooling.

Ventajas y desventajas

Ventajas

Alto rendimiento, control total sobre SQL, eficiencia de memoria con SqlDataReader, base para ORMs.

Desventajas

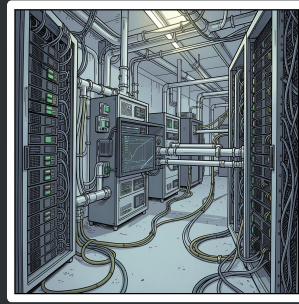
Más código repetitivo (boilerplate), mayor responsabilidad del desarrollador, reader es forward-only, se mantiene la conexión abierta durante la lectura.

Casos de uso comunes



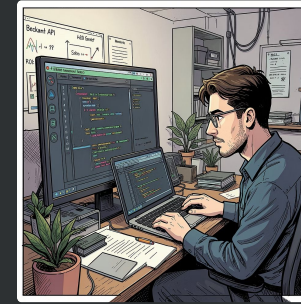
Sistemas de alto rendimiento

Aplicaciones y servicios con consultas intensivas y requisitos de baja latencia (facturación, reportes).



Procesos batch / ETL

Lectura secuencial de millones de registros aprovechando bajo consumo de memoria.



APIs que usan procedimientos

Ejecutar procedimientos almacenados existentes en bases de datos corporativas.



Mejores prácticas y errores a evitar

01

Gestionar recursos

Usar using o try/finally para SqlConnection, SqlCommand y SqlDataReader.

02

Parámetros SQL

Usar SqlParameter en lugar de concatenar para prevenir inyección SQL.

03

Minimizar tiempo abierto

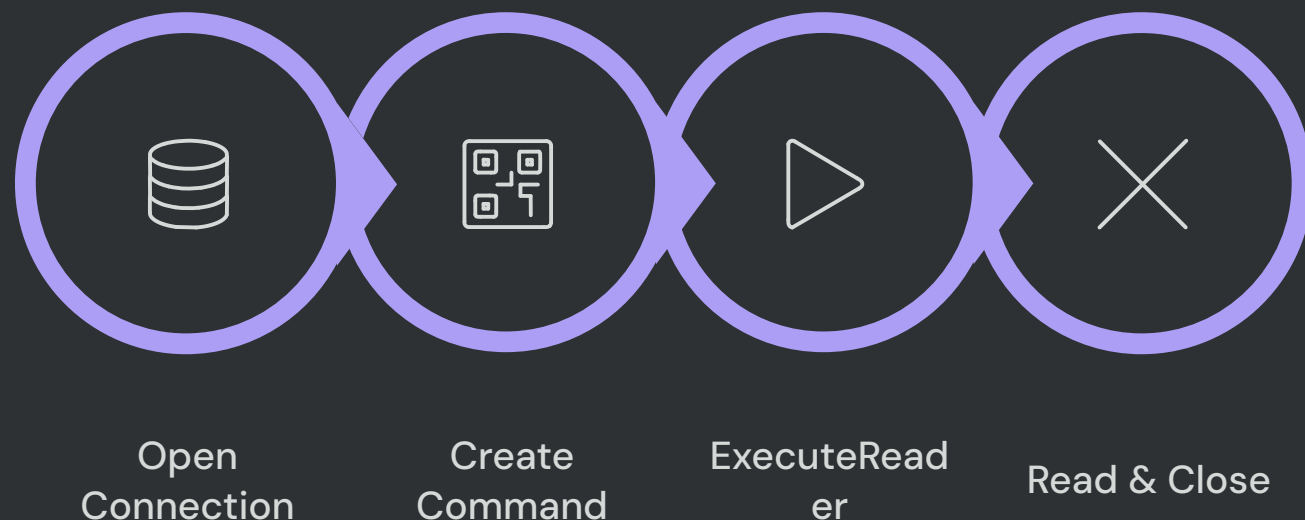
Abrir la conexión justo antes de ejecutar y cerrarla al terminar.

04

Rendimiento

Evitar consultas en bucles (problema N+1); usar tipos de parámetro explícitos y considerar métodos asíncronos.

Relación: SqlConnection → SqlCommand → SqlDataReader



Notas finales

- Los tres trabajan encadenados; ninguno opera de forma aislada.
- SqlDataReader requiere la conexión abierta mientras lee.
- Seguir buenas prácticas garantiza seguridad, rendimiento y escalabilidad.

